

MailWisp 架构纵横分析报告

纵横分析法深度研究报告

研究时间：2026-07-14 | 所属领域：自托管临时邮箱 | 研究对象类型：新产品与技术架构

作者: MailWisp

研究时间：2026-07-14 | 所属领域：自托管临时邮箱 | 研究对象类型：新产品与技术架构

一、一句话定义

MailWisp是一套以可靠收件为核心、面向自托管场景的临时邮箱系统：它用成熟邮件基础设施接住公网SMTP的不确定性，用Go模块化单体控制业务复杂度，用PostgreSQL与可替换内容存储保证数据生命周期，再通过Canonical API和独立Adapter兼容外部生态。

它追求的不是“组件最少”或“跑分最高”这种孤立指标，而是在正确性、安全性、延迟、吞吐、资源、部署和长期维护之间取得可验证的整体最优。

品牌文案已经给出了产品边界：

Fast mail. Zero trace.

来信即现，过时即逝。

“即现”要求收件链路可靠且延迟可预测；“即逝”要求过期、删除、附件、原文、缓存和备份都有一致的生命周期。任何只把邮件显示出来、却无法证明投递确认与删除闭环的方案，都没有完成这句文案承诺。

二、纵向分析：临时邮箱如何走到今天

2.1 最早的问题并不是前端，而是如何接住一封邮件

临时邮箱看起来像一个简单Web应用：生成地址、等待邮件、展示验证码。真正决定系统可靠性的部分却发生在浏览器之外。公网邮件可能重复投递、延迟到达、携带畸形MIME、包含超大附件，也可能在目标服务短暂不可用时重试数小时。SMTP的设计建立在“接受方可以暂时拒绝，由发送方稍后再试”之上，因此收件系统最重要的动作不是解析主题，而是在正确时机回答“这封邮件已经被我可靠接收”。

传统自托管邮件系统长期使用Postfix、Exim等MTA处理公网SMTP、队列、退避、DNS和协议兼容。应用通常通过LMTP、管道或本地投递接口接收邮件。这个分工不新潮，却经过了大量真实互联网环境验证。它把最难伪造的能力——可靠排队与协议边界——留给成熟软件，让业务服务专注邮箱生命周期、邮件解析、检索和API。

早期临时邮箱实现常常走另一条更短的路：应用直接监听SMTP端口，收完邮件立即解析并写数据库。Demo阶段很诱人，因为一个进程就能完成全部流程。但当过载、磁盘故障、数据库超时、解析器卡住或进程重启出现时，系统必须自己实现临时失败、队列、重试、幂等和逐收件人状态。代码行数不一定很多，协议责任却已经接近一个MTA。MailWisp不采用这条路线，不是因为Go无法写SMTP，而是因为重新承担这些责任没有可证明收益。

2.2 从“匿名地址”到“可管理邮箱”

临时邮箱的第一代模型通常把邮箱地址本身当作身份：知道地址就能读邮件，或者创建地址时返回一个长期Token。这个模型极低摩擦，但权限边界很弱。地址一旦泄漏，访问权也随之泄漏；如果Token永久有效且无法撤销，临时产品反而留下长期凭证。

随后出现了两种演进方向。

第一种以mail.tm、DuckMail风格为代表，把临时邮箱建模为带密码的Account。客户端通过Address与Password调用 `/token` 获取JWT，再使用Hydra风格集合读取Messages。这套Contract对自动化客户端友好，也让同一地址可以重新登录。但每个短命邮箱都需要密码KDF、Credential存储和登录防护，系统把正式账户的一部分成本复制到了临时资源上。

第二种以Cloudflare Temp Email和YYDS部分能力为代表，创建邮箱时直接签发地址Token或Temporary Token。它更符合短生命周期资源，也更节省服务端计算，但必须处理Token Hash、Scope、Rotation、撤销和过期。好的实现会把Token只展示一次，数据库仅保存Hash；差的实现会签发永久JWT，甚至把客户端SHA-256结果当作密码。

MailWisp选择第三种更清晰的边界：Canonical Inbox默认使用Passwordless Capability Token；正式用户Account、管理Session与第三方Compatibility Credential分别建模。DuckMail兼容层可以为需要密码登录的Inbox额外建立Argon2id Credential，但不能迫使所有原生Inbox都携带密码。这样兼容外部行为，却不让外部历史设计决定核心领域。

2.3 Cloudflare Temp Email：从极简Worker到平台型产品

`cloudflare_temp_email` 于2023年8月从一个很小的Cloudflare Demo起步。最初链路是Email Routing触发Worker，使用 `postal-mime` 解析邮件，D1保存地址与邮件，Vue前端负责展示。这个选择极大降低了首次部署成本：没有服务器、没有公网SMTP守护进程、没有传统数据库运维。

同一选择也决定了后续演进。项目很快加入Hono、JWT、Admin Panel；2024年因复杂邮件解析问题引入Rust WASM解析器，随后叠加用户注册、Telegram、Webhook、SMTP/IMAP Proxy、S3兼容附件、Passkey、OAuth2、Turnstile和Workers AI。到2025年发布v1.0时，它已经从“临时收件页”演进为Cloudflare上的轻量邮件账户平台。

这段纵向历史揭示了一个重要规律：平台能力会加速功能增长，也会把配置、认证和数据模型逐步绑定到平台。Cloudflare Temp Email当前同时涉及Workers、D1、Email Routing、Pages或Assets，并按功能依赖KV、Workers AI、Rate Limiting、R2/S3和Email Sending。它的功能面很强，但部署问题大量集中在Worker Route、Pages API Base、Catch-all、DNS、CORS和Secret组合上。用户省下了Linux运维，却需要理解一组Cloudflare产品之间的状态关系。

数据模型也留下了演进痕迹。Raw MIME主要存入D1的TEXT/BLOB字段，地址与邮件关联缺少数据库外键，一致性更多依赖应用代码；项目后来出现地址删除后邮件残留的真实修复。大邮件还曾触发D1 `SQLITE_TOOBIG`，Gzip可以缓解部分大小问题，却无法改变单行大对象的架构上限。

MailWisp从这段历史吸收四项经验：保留Raw Source、将Parsed Message作为服务端能力、隔离Webhook/Telegram等Integration、使用Mailpit类E2E验证完整收件链。它拒绝复制Cloudflare绑定、D1 Schema、多套历史Header、永久地址JWT和Raw MIME大BLOB路线。

2.4 DuckMail：稳定Contract与轮询现实

DuckMail 的公开 API 接近 mail.tm 风格，路径简洁，外部客户端容易理解：`/domains`、`/accounts`、`/token`、`/me`、`/messages`、`/sources/{id}`。它使用 Address 与 Password 交换 JWT，消息集合采用 Hydra 结构，`expiresIn` 省略时默认为 24 小时，0 或 -1 代表永久。

这套设计的价值不在功能数量，而在 Contract 清楚。客户端只需要维护一个账户 Token，就可以创建邮箱、列出邮件、读取详情、标记已读和删除资源。对 MailWisp 而言，它是最值得进行“客户端无需修改”兼容测试的对象之一。

它的实时能力则提供了另一种真实教材。仓库中仍能看到 Mercure 相关代码，但当前实现明确提示 Mercure 已不再支持，实际前端采用 1 至 2 秒轮询第一页 Messages。仓库里的 SSE 端点只发送连接事件与心跳，不能证明真实邮件推送。MailWisp 不能因为存在一个 SSE 路径就宣称实时，也不能把固定高频轮询当成最终低负载方案。

更可靠的基线是 Cursor 与 Long Poll：无新消息时连接等待，有消息时返回，断线后客户端用 Cursor 补查。Web UI 可以在同一语义上使用 SSE 改善体验，但事件只负责“唤醒”，事实仍从数据库查询。这样即使实时通道丢事件，也不会丢邮件。

2.5 YYDS：自动化场景推动 API 向前一步

YYDS Mail 的公开 OpenAPI 比 DuckMail 更广。它区分 JWT、`X-API-Key: AC-` 和 Temporary Token，使用 `{success,data,error,errorCode}` Envelope，提供丰富过滤、Webhook、WebSocket 及多种域名能力。

其中最值得 MailWisp 吸收的是 `/messages/next?wait=30`：在一个原子操作中获取最旧未读邮件并标记已读，并发调用者不会获得同一封邮件；没有邮件时可以长轮询等待。这不是为了展示“实时技术”，而是直接解决验证码自动化中的竞争条件。多个 Worker 同时等待同一邮箱时，普通“先列表、再 PATCH 已读”会产生重复消费，而原子领取语义可以在数据库 Transaction 中证明正确。

YYDS 的 Webhook 也展示了成熟边界：HMAC-SHA256 签名、Timestamp、Delivery ID、明确超时、2xx 成功判断、分级重试和投递日志。MailWisp 可以采用同类安全机制，但不需要复制 YYDS 的套餐、支付、OAuth、Passkey 和完整控制台业务。

YYDS 同时说明“兼容”这个词为什么危险。它声称 DuckMail-compatible，却接受后忽略 Password，`/token` 更接近现有 Temporary Token 续签，而不是 DuckMail 的 Address/Password 登录。路径和字段相似不等于语义相同。MailWisp 的兼容声明必须由 Contract Test 定义 Supported、Partially Supported 和 Unsupported，不能使用模糊的“基本兼容”。

2.6 旧TempMail：功能已经很多，边界已经开始失控

本地旧项目已经具备 Go API、Go LMTP、Postfix、PostgreSQL、PgBouncer、Redis 和 Nginx 前端，功能覆盖域名、邮箱、邮件、验证码、链接、Token、管理员、健康检查和清理任务。它不是一个应被轻易丢弃的 Demo，而是一份真实需求与迁移约束的证据库。

问题集中在结构演进。`store/postgres.go`、`ingress/server.go` 和 `store/tokens.go` 已经成为超大文件，DNS 查询进入 PostgreSQL Store，`main.go` 同时负责 HTTP、LMTP、Job、Key 文件和 Lifecycle。前端单一 `app.js` 超过 4000 行、`style.css` 超过 8000 行，大量字符串 HTML、Inline Event 和多代 Dark Theme 覆盖相互叠加。

这些问题并不说明Go、PostgreSQL或原生前端天然错误。真正原因是边界没有随着功能增长被重新组织。旧项目最有价值的部分是生产行为、数据库Invariant、LMTP样本、API错误语义和真实迁移数据；最不值得复用的是已经混合职责的文件结构。MaiWisp对旧代码的态度应当像对待一个运行多年的协议实现：保留证据，逐项提取，任何复制都必须经过测试。

三、横向分析：当前方案分别活成了什么样

3.1 五种路线的核心对比

维度	旧TempMail	Cloudflare Temp Email	DuckMail	YYDS	MaiWisp目标
主 要 用户	单机自托管	Cloudflare用户	公共API客户端	自动化与平台用户	高质量自托管与API集成
公 网 收件	Postfix→LMTP	Email Routing→Worker	官方资料未完整公开	官方资料未完整公开	Postfix→LMTP
应 用 形态	Go服务但职责混合	Workers 多 产 品 编排	Web/API产品	平台型API	Go模块化单体
持 久 数据	PostgreSQL	D1	服务端实现未完整公开	服务端实现未完整公开	PostgreSQL
Raw MIME	DB与文件并存	D1 TEXT/BLOB	Source API	Source API	内 容 寻 址 文 件/S3 Adapter
临 时 状态	Redis	KV/Rate Limiting	未公开	平台内部能力	进程内优先，Redis可选
原 生 认证	Token 与 管理 Key	多套JWT/Header	Password→JWT	JWT/API Token	Key/Temp Token + Session
实时	轮询/接口	多种集成	1-2秒轮询	Long Poll/WebSocket/Webhook	Long Poll 基线，SSE优化
部 署 复 杂 度	多 容 器 与 双 Nginx	多Cloudflare产品配置	托管服务	托管服务	单机Reference Profile
主 要 风险	边界腐化	平台锁定与历史包袱	实时与公开配额不明	产品面过宽	新项目需证明每项选择

3.2 “最少组件”并不自动等于低占用

一个Go进程直接实现SMTP、HTTP、队列、数据库和文件管理，看起来组件最少，却会把成熟MTA的状态机搬进应用代码。代码中的队列同样占内存，崩溃恢复同样需要磁盘格式，协议漏洞同样需要持续修复。减少进程数量不等于减少系统总复杂度。

反过来，引入Redis、PgBouncer、RabbitMQ和对象存储也不自动等于专业。单机单应用如果只有一个pgx Pool，PgBouncer增加一次网络转发和另一套连接语义；如果限流只服务一个进程，Redis增加一个故障点；如果后台任务只在一个实例运行，RabbitMQ增加队列持久化与运维，却没有独立扩容收益。

MailWisp Reference Profile应当保留不可替代的四类责任：公网SMTP与队列、应用业务、事实数据库、HTTP TLS入口。Raw MIME需要独立于关系数据库的大对象存储语义，但在单机上可以由本地文件系统实现，不必强制部署MinIO。Redis、PgBouncer、独立消息队列和S3服务都必须等到多实例或测量证据出现后再进入Extended Profile。

3.3 模块化单体与微服务

MailWisp的领域确实存在可拆边界：Ingress、Parser、Mailbox、Message、Auth、Webhook和Cleanup。但“存在边界”不等于“需要网络”。一名维护者、一个Reference节点、没有独立团队和没有独立扩容数据时，把这些模块拆成微服务会立即引入RPC Contract、Service Discovery、跨服务Tracing、部署排序和分布式失败。

模块化单体保留代码边界，使用进程内调用完成事务。一个Go二进制可以提供不同Role：`serve`同时运行API、LMTP和Worker；Extended Profile再按 `api`、`ingress`、`worker` 拆进程。拆分后仍共享同一Domain与Repository Contract，不需要提前制造内部HTTP或gRPC。

这种设计不是“以后也许能扩展”的空话。它有明确演进条件：LMTP解析CPU影响API尾延迟、Webhook重试需要独立资源、多个API Replica导致连接压力、或单机存储无法满足容量。条件出现前保持同进程；出现后使用同一二进制分Role，并将本地Raw Storage替换为S3 Adapter。演进路径由代码边界支撑，不由预先部署一堆空服务支撑。

3.4 PostgreSQL、Redis与Raw Storage

PostgreSQL 适合保存Inbox、Address、Credential、Message Metadata、状态、Retention、Webhook Outbox和审计事实。它提供Transaction、外键、唯一约束、`FOR UPDATE SKIP LOCKED`、Advisory Lock和成熟备份工具。这些能力可以同时支撑原子领取下一封邮件、后台任务单实例协调和Transactional Outbox。

PostgreSQL不适合无上限承载所有Raw MIME和附件大BLOB。问题不只是单行大小，还包括WAL膨胀、备份时间、Vacuum、Cache污染和删除后空间回收。MailWisp应先把Raw Source流式写入同文件系统的Staging区域，计算Hash并 `fsync`，再原子Rename到内容寻址路径；数据库Transaction只在文件耐久后创建Message与Content Reference。数据库失败会留下可扫描Orphan，文件失败则不确认LMTP成功。后台Parser读取Raw Source，产生有界的Text、HTML、Header和Attachment Metadata。

Redis在Reference Profile中不是必需事实源。进程内有界Token Bucket可以处理单进程瞬时限流，PostgreSQL保存必须持久的日配额；Long Poll唤醒可用进程内Event Hub，Extended Profile可使用PostgreSQL `LISTEN/NOTIFY` 作为无持久语义的唤醒通道。只有多Replica热点限流或通知压力得到测量后，Redis Adapter才启用。这样Reference部署少一个服务，也没有假装Redis“高性能”就一定更优。

3.5 前端：原生最小、Vue最平衡

四套同功能Technical Spike给出了可比较数据。原生TypeScript总Gzip约3.7 KB，Svelte约18.1 KB，Vue约27.6 KB，React约62.3 KB。原生方案在网络上明显最小，却需要最多手写DOM、Escape、事件重绑定和生命周期代码，已经出现整页 `innerHTML` 重渲染。旧项目的大文件历史说明这种成本会随着管理功能放大。

Vue比Svelte多约9.5 KB Gzip，但提供成熟Component模型、TypeScript、vue-i18n、测试和Accessibility生态。对Content Hash与Brotli缓存后的控制台，这个一次性差异不应压过长期维护。React没有为MailWisp提供独占能力，当前切片Runtime最大；Svelte保留为真实性能瓶颈出现后的替代候选。

更关键的证据来自测试过程：四套方案首次都因Favicon 404触发Console Error；Mist主题Muted文字首次只有4.1:1对比度，低于WCAG AA 4.5:1。修复共享Semantic Token后，axe扫描全部通过。这说明主题系统的价值不在“24套”，而在所有Preset共享完整Token Contract并经过自动对比度、Keyboard和截图回归门禁。

3.6 Nginx与Caddy：仍不能凭偏好决定

Caddy的优势是自动HTTPS、配置简洁和安全默认值；Nginx的优势是生态成熟、资源行为可预测、与现有运维经验及外部证书工具配合广泛。对普通Web应用，Caddy常常降低个人部署成本；对同时需要HTTPS与SMTP TLS的邮件系统，证书必须安全地提供给Web入口和Postfix，单纯比较一份Web配置并不充分。

因此MailWisp尚未把Nginx或Caddy写成最终结论。Technical Spike必须在同一Linux环境比较：首次签发、续签、Postfix证书共享、零停机Reload、OCSP、日志轮转、配置行数、空闲内存、故障恢复和容器/宿主机边界。没有这组证据，任何“Caddy更现代”或“Nginx性能更高”的判断都属于偏好，不属于架构。

四、横纵交汇：MailWisp应如何避免重走旧路

4.1 今天的每个优势，都应有历史来源

MailWisp选择Postfix，不是保守，而是从SMTP长期演进和旧项目可用链路中确认：公网队列与重试已经有成熟Owner。选择LMTP，是为了让应用可以对每个Recipient返回明确状态，并在Durable Persistence前拒绝成功。

选择Canonical API加Adapter，来自Cloudflare Temp Email多套Header、DuckMail密码账户与YYDS Temporary Token之间的真实语义冲突。把任何一家直接当核心API，都会让另两家兼容变成领域污染。

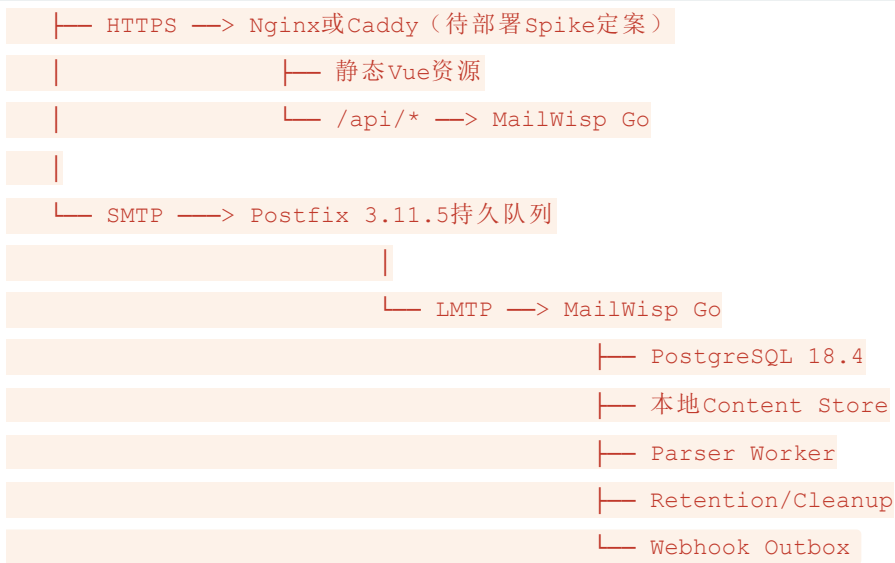
选择Raw/Parsed分离，来自Cloudflare大邮件与重新解析需求，也来自旧项目同时保存Raw与Parsed字段的生产经验。选择本地内容寻址文件作为Reference Storage，来自关系数据库大BLOB的WAL与备份成本，同时尊重个人服务器不应被强制部署MinIO。

选择Vue，不来自流行度，而来自同功能构建体积、专属代码、i18n、主题和axe测试。选择模块化单体，也不来自“微服务不好”，而来自当前只有一名Owner、单Reference节点、没有独立扩缩容证据。

4.2 当前推荐架构

Internet

|



Reference Profile使用一个MailWisp进程承担HTTP、LMTP、后台解析和清理，但内部Queue全部有界，每个Worker有Owner、取消和过载行为。LMTP接收Raw Source并可靠落盘、写入数据库后才返回成功；解析、验证码提取、Webhook和通知在持久化之后进行，任何Integration失败都不能让已接收邮件消失。

PostgreSQL是唯一业务事实源。本地Content Store保存Raw MIME和需要独立生命周期的大附件。Redis默认不部署；PgBouncer默认不部署；不使用Kafka、RabbitMQ、Kubernetes、Service Mesh或内部RPC。Extended Profile在证据出现时可以拆Role、启用S3 Storage、Redis与PgBouncer，但这些不是Reference Profile的前置成本。

Canonical API使用 `/api/v1`、RFC3339 UTC、稳定Error Code和Request ID。Inbox默认签发256-bit以上随机Capability Token，服务端只保存Hash与展示前缀。Web Console优先使用Secure、HttpOnly、SameSite Cookie。DuckMail、YYDS和Cloudflare Temp Email通过独立路由与Presenter兼容；Compatibility可以关闭，Legacy Root Path默认关闭。

Web实时能力以Long Poll和Cursor为正确性基线，SSE用于界面优化。Webhook采用Transactional Outbox、HMAC、Timestamp、Delivery ID、有限重试与投递日志。WebSocket只有在双向交互或连接规模证明收益后再加入。

4.3 这套架构不是“最终答案”，而是当前证据下的最小充分解

仍未完成的关键验证包括：

- Postfix到Go LMTP的真实过载、临时失败、重复投递和重启恢复。
- Raw MIME本地Content Store的断电一致性、Orphan扫描、备份与恢复。
- PostgreSQL 18.4对旧Dump迁移、索引和查询计划的兼容。
- Nginx与Caddy在HTTPS和SMTP证书共享场景的部署实测。
- MIME Parser对真实Corpus、恶意嵌套、超大Header和附件的Fuzz与内存上限。
- Canonical、DuckMail、YYDS与Cloudflare Adapter的黑盒Contract Test。
- 单机Reference Profile的空闲内存、收件延迟、API P95/P99和过载曲线。

这些未验证项不会被文档措辞包装成已完成。任何一项Spike证明候选不可接受，就更新ADR并替换方案。

4.4 三个未来剧本

最可能剧本：成为可靠、克制的自托管工具

MailWisp按Reference Profile完成Postfix、Go、PostgreSQL和本地Content Store闭环，Vue控制台覆盖中文、英文、Light、Dark、Follow System与少量高质量Preset。原生API稳定，DuckMail兼容优先落地，YYDS与Cloudflare按价值逐步补齐。单机能够承受个人与小团队的真实流量，维护者可以独立备份、恢复、升级和排障。

这个剧本成功的关键不是功能数量，而是每次扩展都守住数据生命周期和边界。它比Cloudflare Temp Email少一些平台功能，比YYDS少商业体系，但自托管体验更完整，架构更可控。

最危险剧本：为了“专业”重新堆回复杂度

项目在没有负载证据时加入Redis、PgBouncer、消息队列、微服务、WebSocket和多套存储，又同时追求三家API百分百兼容、24套主题、IMAP、SMTP发信与完整用户平台。每个组件单独合理，组合后却超过一名维护者的验证能力。最终Scaffold很多，Migration、恢复、过载和Contract Test没有闭合。

这个风险与旧项目巨型文件相反，却来自同一个根源：边界增长快于验证能力。MailWisp必须允许明确舍弃功能，不能把“支持更多”误认为“架构更优”。

最乐观剧本：单机Reference与可控扩展同时成立

模块化单体在代码层保持稳定，Reference Profile维持少量服务；当真实用户需要更高吞吐时，同一Go二进制按Role拆分，Raw Storage切换S3，PostgreSQL Outbox与Advisory Lock继续保持一致语义。兼容层通过Contract Fixture独立演进，核心Domain不随外部API变化。

此时MailWisp既没有为了早期简洁封死扩展，也没有为了未来想象提前支付分布式成本。它真正实现的不是“无限扩展”，而是从单机到多实例的每一步都有触发条件、测试和回滚路径。

五、当前决策与舍弃清单

主题	当前判断	证据状态
Backend	Go 1.26.5模块化单体	编译、Test、Race、Vet、govulncheck已通过
HTTP Router	优先标准库 <code>net/http</code>	Go现代路由能力足够，业务Vertical Slice继续验证
Public SMTP	Postfix 3.11.5	成熟队列与旧项目经验支持，LMTP E2E待补
Database	PostgreSQL 18.4 + pgx	版本来源确认，旧Dump与Repository Spike待补
Raw Storage	本地内容寻址文件 + S3 Adapter	架构候选，断电与恢复Spike待补
Redis	Reference默认不启用	单进程暂无可替代收益，Extended保留Adapter

主题	当前判断	证据状态
PgBouncer	Reference不启用	单Pool无连接压力证据
Async Queue	PostgreSQL Outbox/有界Worker	不引入Broker，故障与吞吐待测
Frontend	Vue 3.5.39 + TypeScript 6.0.3	四方案同功能Spike与axe已通过
Realtime	Long Poll基线，SSE优化	吸收YYDS原子领取，拒绝假实时
Compatibility	Canonical API +独立Adapter	三方Contract已初步核验，黑盒测试待补
Nginx/Caddy	暂不定案	必须完成证书共享和运维Spike
Microservices	舍弃	当前无团队、部署和独立扩容收益
Kafka/RabbitMQ	舍弃	PostgreSQL Outbox足以覆盖当前故障模型
MinIO强制依赖	舍弃	本地Content Store更符合Reference Profile
Rust	舍弃	项目统一Go，MIME能力通过Go Corpus验证
React主方案	舍弃	同功能Runtime最大且无独占收益
24套主题KPI	舍弃	主题数量服从Token完整性与Accessibility

六、信息来源

官方与一手来源

- Go Downloads：https://go.dev/dl/?mode=json（访问日期：2026-07-14）
- PostgreSQL版本信息：https://www.postgresql.org/versions.json（访问日期：2026-07-14）
- Postfix 3.11.5公告：https://www.postfix.org/announcements/postfix-3.11.5.html（访问日期：2026-07-14）
- Nginx下载与版本：https://nginx.org/en/download.html（访问日期：2026-07-14）
- Caddy Releases：https://github.com/caddyserver/caddy/releases/tag/v2.11.4（访问日期：2026-07-14）
- Cloudflare Temp Email仓库：https://github.com/dreamhunter2333/cloudflare_temp_email（访问日期：2026-07-14）
- Cloudflare Temp Email Releases：https://github.com/dreamhunter2333/cloudflare_temp_email/releases（访问日期：2026-07-14）

- Cloudflare Temp Email 大 邮 件 Issue :
https://github.com/dreamhunter2333/cloudflare_temp_email/issues/823 (访问日期: 2026-07-14)
- Cloudflare Temp Email 部 署 Issue :
https://github.com/dreamhunter2333/cloudflare_temp_email/issues/607 (访问日期: 2026-07-14)
- DuckMail API文档: <https://www.duckmail.sbs/en/api-docs> (访问日期: 2026-07-14)
- DuckMail官方仓库: <https://github.com/MoonWeSif/DuckMail> (访问日期: 2026-07-14)
- YYDS Mail文档: <https://vip.215.im/docs> (访问日期: 2026-07-14)
- YYDS OpenAPI: <https://maliapi.215.im/v1/openapi.json> (访问日期: 2026-07-14)
- YYDS错误码: <https://maliapi.215.im/v1/error-codes> (访问日期: 2026-07-14)

本地工程证据

- `../tempmail` 只读源代码、Migration、Compose与API实现盘点。
- `spikes/frontend` 四套同功能前端实现、Production Build、Playwright与axe-core结果。
- MailWisp Go 1.26.5下的Test、Race、Vet与 `govulncheck v1.6.0` 验证结果。
- `docs/research/03-cloudflare-temp-email.md` 完整Git历史、源码与Issue研究。
- `docs/research/04-api-compatibility-preliminary.md` 第三方API Contract矩阵。

七、方法论说明

本报告使用纵横分析法：纵向追踪临时邮箱从传统MTA分工、匿名地址、密码账户、Capability Token到平台型API的演进；横向比较旧TempMail、Cloudflare Temp Email、DuckMail与YYDS的当前结构，再将两条轴交汇为MailWisp的架构判断。所有“最优”结论均受当前约束和证据范围限制，可由后续Technical Spike、Benchmark、Contract Test与恢复演练推翻。